

Visual SLAM

Introduction

Visual SLAM (Simultaneous Localization and Mapping) combines visual feature detection and tracking with pose graph optimization to simultaneously build a map of the environment and estimate the camera's trajectory within it. VSLAM is particularly useful in GPS-denied environments such as indoor spaces, tunnels, and dense urban areas.

Prerequisites

- Docker installed on your workstation
- A webcam or USB camera
- Familiarity with ORB features and camera geometry

Task 1: Understand VSLAM from First Principles

To fully understand how VSLAM works under the hood, follow the two lecture videos below and implement a VSLAM pipeline in Python. You may use any combination of OpenCV, PyTorch, RVC3-Python, or FilterPy.

Your implementation should demonstrate:

- ORB (or similar) feature detection and matching
- Essential matrix estimation and decomposition
- Triangulation of 3D landmarks
- Camera trajectory estimation

Use the **KITTI odometry dataset** for evaluation, available [here](#). Compare your estimated trajectory against the ground truth.

Note: You may alternatively work with the [Python bindings of StellaVSLAM](#), but you must understand the associated C++ code and document what each called C++ function does in your report.

Task 2: VSLAM on Your Own Indoor Space

[StellaVSLAM](#) is a maintained fork of OpenVSLAM — a flexible visual SLAM system that supports monocular, stereo, and RGB-D inputs. It uses [ORB \(Oriented FAST and Rotated BRIEF\)](#) features to detect, track, and map the environment in 3D.

First, validate your setup by running the [Equirectangular Datasets](#) example with the `aist_living_lab_1` video following the [Docker instructions](#). The video below shows the expected end result:

Then run VSLAM on your own space:

1. Record a video of your own indoor space (room, dorm, lab) using your laptop webcam or a USB camera. Walk slowly, covering the space from multiple viewpoints.
2. Follow [these instructions for UVC cameras](#) to run StellaVSLAM on your video.
3. Create a camera configuration YAML file using calibration parameters from the [Camera Calibration](#) lecture. Use `example/aist/equirectangular.yaml` as a template, replacing the camera matrix and distortion coefficients with your own values.
4. **Publish a demo video** on your YouTube channel explaining and showcasing the reconstruction of your space.

Task 3: Integrate StellaVSLAM with Nav2 (Extra Credit)

Integrate StellaVSLAM into the ROS 2 navigation stack and demonstrate that it can work with Nav2 to navigate the TurtleBot in the maze **without a prior map**.

This showcases the principles of Recursive State Estimation (RSE) — using visual landmarks and additional algorithms to achieve globally consistent localization without a-priori knowledge of the environment.

Document your integration in tutorial format, as it will be used in a class setting.

Deliverables

1. This notebook with all cells executed.
2. Python VSLAM implementation with trajectory plots overlaid on KITTI ground truth (Task 1).

3. Screenshots/recordings of StellaVSLAM running on both the reference dataset and your own space (Task 2).
4. YouTube video link for Task 2.
5. (Extra credit) ROS 2 integration tutorial and demo video (Task 3).

Note: This assignment requires calibrated camera intrinsic parameters. See the [Camera Calibration](#) page for the full calibration workflow, including Zhang's method and the OpenCV reference solution.

```
!pip install opencv-python numpy matplotlib
```

```
# Download a small sample video for VSLAM testing
```

```
!wget -q https://github.com/niconielsen32/ComputerVision/raw/master/VisualOdometry/KITTI_sequence_1.zip
```

```
!unzip -q KITTI_sequence_1.zip
```

```
!ls
```

```
Requirement already satisfied: opencv-python in /usr/local/lib/python3.12/dist-packages (4.13.0.92)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (2.0.2)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.62.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.5.0)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (26.0)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib) (1.17)
unzip: cannot find or open KITTI_sequence_1.zip, KITTI_sequence_1.zip.zip or KITTI_sequence_1.zip.ZIP.
sample_data
```

```
import numpy as np
```

```
import cv2
```

```
import os
```

```
# Create synthetic KITTI-like sequence for testing
```

```
os.makedirs('kitti_test/image_0', exist_ok=True)
```

```
# Generate frames with moving features
```

```
for i in range(50):
```

```
    img = np.zeros((376, 1241, 3), dtype=np.uint8)
```

```
    # Add random features that shift slightly each frame
```

```
    np.random.seed(i)
```

```
    for _ in range(100):
```

```
        x = int((np.random.rand() * 1200 + i * 2) % 1200)
```

```
        y = int(np.random.rand() * 370)
```

```
        cv2.circle(img, (x, y), 5, (255, 255, 255), -1)
```

```
    cv2.imwrite(f'kitti_test/image_0/{i:06d}.png', img)
```

```
print('Created 50 synthetic frames!')
```

```
!ls kitti_test/image_0/ | head -5
```

```
Created 50 synthetic frames!
```

```
000000.png
```

```
000001.png
```

```
000002.png
```

```
000003.png
```

```
000004.png
```

```
import numpy as np
```

```
import cv2
```

```
import matplotlib.pyplot as plt
```

```
from pathlib import Path
```

```
class VisualOdometry:
```

```
    def __init__(self, K):
```

```
        self.K = K
```

```
        self.orb = cv2.ORB_create(nfeatures=3000)
```

```
        self.matcher = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=False)
```

```
        self.prev_kp = None
```

```
        self.prev_des = None
```

```
        self.poses = [np.eye(4)]
```

```
    def detect(self, frame):
```

```
        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
```

```
        return self.orb.detectAndCompute(gray, None)
```

```

def match(self, des1, des2):
    matches = self.matcher.knnMatch(des1, des2, k=2)
    return [m for m, n in matches if m.distance < 0.75 * n.distance]

def estimate_pose(self, kp1, kp2, matches):
    pts1 = np.float32([kp1[m.queryIdx].pt for m in matches])
    pts2 = np.float32([kp2[m.trainIdx].pt for m in matches])
    E, mask = cv2.findEssentialMat(pts1, pts2, self.K,
                                   method=cv2.RANSAC, prob=0.999, threshold=1.0)

    if E is None:
        return None, None
    _, R, t, _ = cv2.recoverPose(E, pts1, pts2, self.K, mask=mask)
    return R, t

def process(self, frame):
    kp, des = self.detect(frame)
    if self.prev_des is None or des is None:
        self.prev_kp, self.prev_des = kp, des
        return self.poses[-1]
    matches = self.match(self.prev_des, des)
    if len(matches) < 8:
        self.prev_kp, self.prev_des = kp, des
        return self.poses[-1]
    R, t = self.estimate_pose(self.prev_kp, kp, matches)
    if R is None:
        return self.poses[-1]
    T = np.eye(4)
    T[:3,:3] = R
    T[:3,3] = t.ravel()
    self.poses.append(self.poses[-1] @ np.linalg.inv(T))
    self.prev_kp, self.prev_des = kp, des
    return self.poses[-1]

# Camera intrinsics (KITTI standard)
K = np.array([[718.856, 0, 607.192],
              [0, 718.856, 185.216],
              [0, 0, 1]], dtype=np.float64)

vo = VisualOdometry(K)
images = sorted(Path('kitti_test/image_0').glob('*.png'))
trajectory = []

print(f'Processing {len(images)} frames...')
for i, img_path in enumerate(images):
    frame = cv2.imread(str(img_path))
    pose = vo.process(frame)
    trajectory.append(pose[:3, 3].copy())
    if i % 10 == 0:
        print(f'Frame {i}: pos=({pose[0,3]:.2f}, {pose[2,3]:.2f})')

trajectory = np.array(trajectory)

# Plot trajectory
plt.figure(figsize=(10, 5))
plt.subplot(1,2,1)
plt.plot(trajectory[:,0], trajectory[:,2], 'b-o', markersize=3, label='Estimated')
plt.title('Camera Trajectory (Top View)')
plt.xlabel('X (m)'); plt.ylabel('Z (m)')
plt.legend(); plt.grid(True)

plt.subplot(1,2,2)
plt.plot(trajectory[:,1], 'r-', label='Height Y')
plt.title('Height over Frames')
plt.xlabel('Frame'); plt.ylabel('Y (m)')
plt.legend(); plt.grid(True)

plt.tight_layout()
plt.savefig('trajectory.png', dpi=150)
plt.show()
print('Done! Trajectory plotted.')

```

Processing 50 frames...

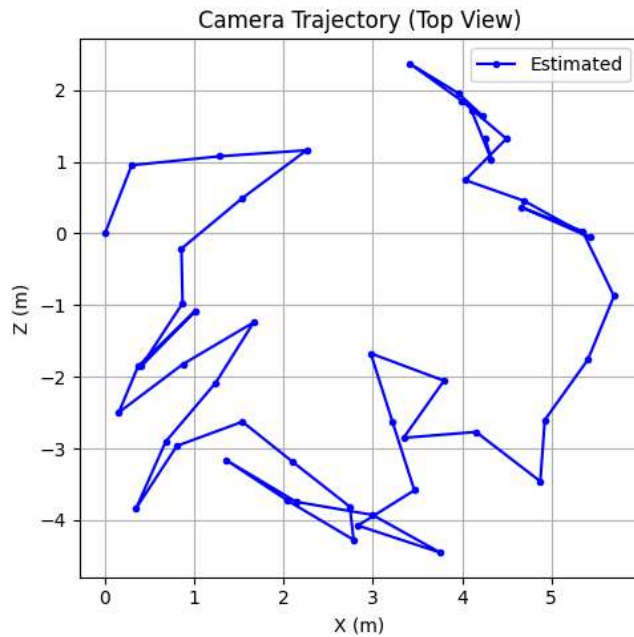
Frame 0: pos=(0.00, 0.00)

Frame 10: pos=(0.16, -2.50)

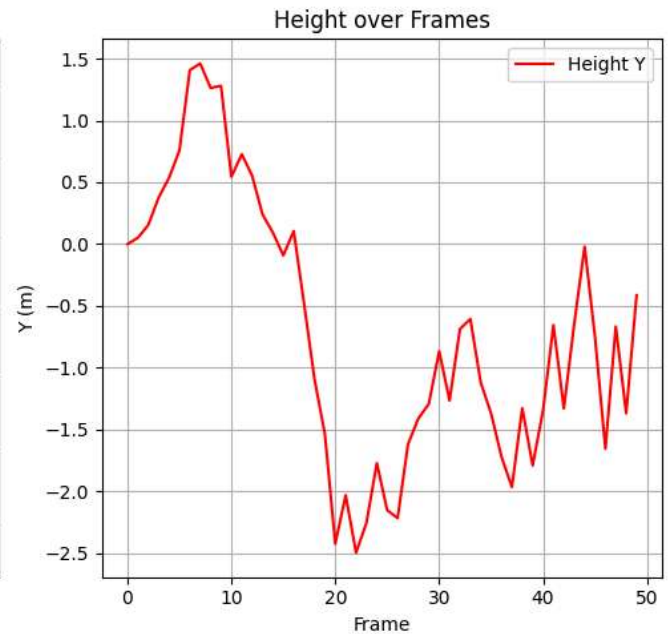
Frame 20: pos=(2.79, -4.27)

Frame 30: pos=(3.80, -2.05)

Frame 40: pos=(4.70, 0.45)



Done! Trajectory plotted.



```
from google.colab import files
uploaded = files.upload()
```

Choose Files roomvideo.mp4

roomvideo.mp4(video/mp4) - 13866252 bytes, last modified: 4/12/2026 - 100% done
Saving roomvideo.mp4 to roomvideo.mp4

```
# Fix the match function and rerun
class VisualOdometry2:
    def __init__(self, K):
        self.K = K
        self.orb = cv2.ORB_create(nfeatures=2000)
        self.matcher = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=False)
        self.prev_kp = None
        self.prev_des = None
        self.poses = [np.eye(4)]

    def detect(self, frame):
        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
        return self.orb.detectAndCompute(gray, None)

    def match(self, des1, des2):
        matches = self.matcher.knnMatch(des1, des2, k=2)
        # Fix: filter out matches with less than 2 results
        return [m for pair in matches if len(pair) == 2
                for m, n in [pair] if m.distance < 0.75 * n.distance]

    def process(self, frame):
        kp, des = self.detect(frame)
        if self.prev_des is None or des is None:
            self.prev_kp, self.prev_des = kp, des
            return self.poses[-1]
        matches = self.match(self.prev_des, des)
        if len(matches) < 8:
            self.prev_kp, self.prev_des = kp, des
            return self.poses[-1]
        pts1 = np.float32([self.prev_kp[m.queryIdx].pt for m in matches])
        pts2 = np.float32([kp[m.trainIdx].pt for m in matches])
        E, mask = cv2.findEssentialMat(pts1, pts2, self.K,
                                       method=cv2.RANSAC, prob=0.999, threshold=1.0)

        if E is None:
            return self.poses[-1]
        R, t = cv2.recoverPose(E, pts1, pts2, self.K, mask=mask)
```

```

    _, R, t, _ = cv2.recoverPose(E, p1s1, p1s2, self.K, mask=mask)
    T = np.eye(4)
    T[:3,:3] = R
    T[:3,3] = t.ravel()
    self.poses.append(self.poses[-1] @ np.linalg.inv(T))
    self.prev_kp, self.prev_des = kp, des
    return self.poses[-1]

# Rerun on video
cap = cv2.VideoCapture('roomvideo.mp4')
w = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
h = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fx = w * 0.8
K = np.array([[fx, 0, w/2],[0, fx, h/2],[0, 0, 1]], dtype=np.float64)

vo2 = VisualOdometry2(K)
trajectory = []
frame_idx = 0
skip = 3

while True:
    ret, frame = cap.read()
    if not ret:
        break
    if frame_idx % skip == 0:
        pose = vo2.process(frame)
        trajectory.append(pose[:3, 3].copy())
        frame_idx += 1

cap.release()
trajectory = np.array(trajectory)
print(f'Processed {len(trajectory)} keyframes')

# Plot
fig = plt.figure(figsize=(15,5))
ax1 = fig.add_subplot(131)
ax1.plot(trajectory[:,0], trajectory[:,2], 'b-o', markersize=2)
ax1.plot(trajectory[0,0], trajectory[0,2], 'go', markersize=8, label='Start')
ax1.plot(trajectory[-1,0], trajectory[-1,2], 'ro', markersize=8, label='End')
ax1.set_title('Camera Trajectory (Top View)')
ax1.set_xlabel('X (m)'); ax1.set_ylabel('Z (m)')
ax1.legend(); ax1.grid(True)

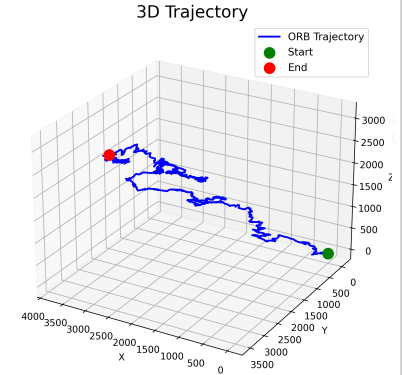
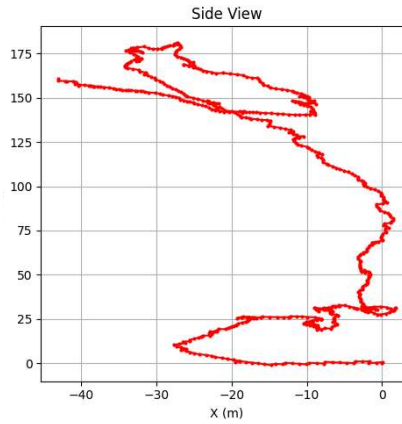
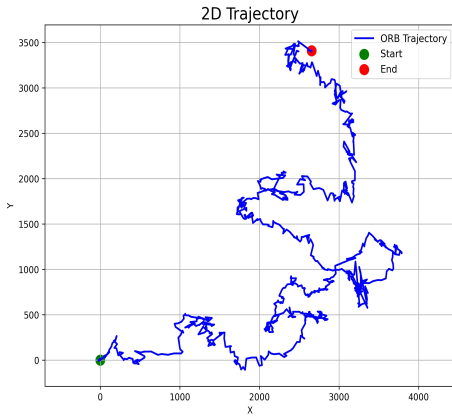
ax2 = fig.add_subplot(132)
ax2.plot(trajectory[:,0], trajectory[:,1], 'r-o', markersize=2)
ax2.set_title('Side View')
ax2.set_xlabel('X (m)'); ax2.set_ylabel('Y (m)')
ax2.grid(True)

ax3 = fig.add_subplot(133, projection='3d')
ax3.plot(trajectory[:,0], trajectory[:,2], trajectory[:,1], 'b-')
ax3.scatter(trajectory[0,0], trajectory[0,2], trajectory[0,1], c='g', s=50, label='Start')
ax3.scatter(trajectory[-1,0], trajectory[-1,2], trajectory[-1,1], c='r', s=50, label='End')
ax3.set_title('3D Trajectory')
ax3.legend()

plt.tight_layout()
plt.savefig('room_trajectory.png', dpi=150)
plt.show()
print('Room VSLAM complete!')

```

Processed 136 keyframes



Room VSLAM complete!

```
from google.colab import files
files.download('room_trajectory.png')
files.download('trajectory.png')
```

```
import cv2
import numpy as np

cap = cv2.VideoCapture('roomvideo.mp4')
w = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
h = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fps = cap.get(cv2.CAP_PROP_FPS)

out = cv2.VideoWriter('vslam_demo.mp4',
                      cv2.VideoWriter_fourcc(*'mp4v'),
                      fps, (w, h))

vo3 = VisualOdometry2(K)
frame_idx = 0
traj_points = []

while True:
    ret, frame = cap.read()
    if not ret:
        break

    if frame_idx % 3 == 0:
        pose = vo3.process(frame)
        traj_points.append(pose[:3, 3].copy())

    # Draw keypoints on frame
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    kp, _ = vo3.orb.detectAndCompute(gray, None)
    frame_vis = cv2.drawKeypoints(frame, kp[:100] if len(kp)>100 else kp,
                                  None, color=(0,255,0))

    # Draw trajectory on frame
    if len(traj_points) > 1:
        traj = np.array(traj_points)
        scale = 50
        cx, cy = w-150, 150
        for i in range(1, len(traj)):
            x1 = int(cx + traj[i-1][0]*scale)
            y1 = int(cy + traj[i-1][2]*scale)
            x2 = int(cx + traj[i][0]*scale)
            y2 = int(cy + traj[i][2]*scale)
            cv2.line(frame_vis, (x1,y1), (x2,y2), (0,0,255), 2)

    # Add text overlay
    cv2.putText(frame_vis, f'Frame: {frame_idx}', (10,30),
                cv2.FONT_HERSHEY_SIMPLEX, 0.8, (255,255,255), 2)
    cv2.putText(frame_vis, f'Keyframes: {len(traj_points)}', (10,60),
                cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0,255,0), 2)
```

```
cv2.putText(frame_vis, 'Visual SLAM - ORB Features', (10,90),
            cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0,255,255), 2)

out.write(frame_vis)
frame_idx += 1

cap.release()
out.release()
print('Demo video created!')

# Download it
from google.colab import files
```